

Manual técnico

Guía para programadores · Finax

Versión 1.0 · Octubre 2026

Contenido

#	Sección
1	Resumen y tecnologías
2	Arquitectura
3	Estructura del repositorio
4	Base de datos (diagrama y diccionario de datos)
5	Guía de clonación y despliegue desde GitHub
6	API REST
7	Lector de QR / código de barras
8	Seguridad
9	Manejo de errores
10	Mi mes en espejo (lógica)
11	Limitaciones y mejoras

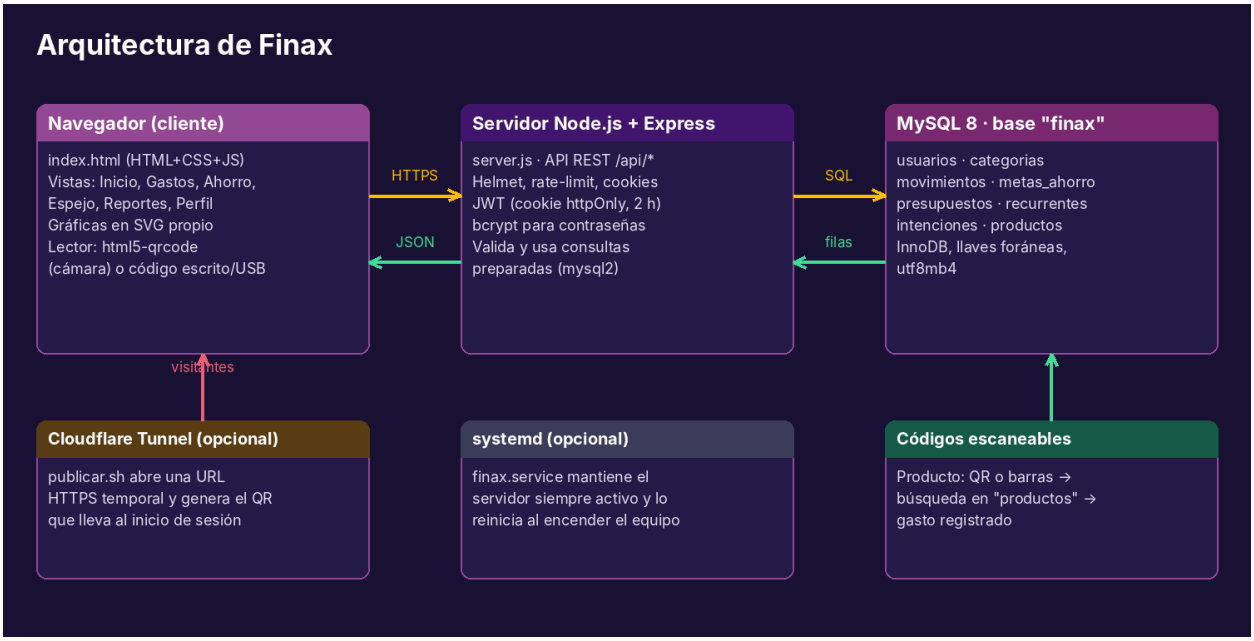
1. Resumen y tecnologías

Finax es una aplicación web de finanzas personales: el usuario inicia sesión, registra ingresos y gastos, fija presupuestos, crea metas de ahorro y revisa reportes. Además tiene **Mi mes en espejo** (compara lo que la persona se propuso con lo que hizo) y un **lector de QR / código de barras** que registra gastos al escanear un producto.

Capa	Tecnología	Para qué
Servidor	Node.js 20 + Express	API REST y entrega de la página
Base de datos	MySQL 8 (InnoDB, utf8mb4)	Almacenamiento relacional
Conector	`mysql2/promise`	Consultas preparadas con pool de conexiones
Login	`bcryptjs` + `jsonwebtoken`	Hash de contraseñas y sesión en cookie httpOnly
Protección	`helmet`, `express-rate-limit`	Cabeceras seguras y límite de intentos
Interfaz	HTML + CSS + JavaScript puro	Una sola página, gráficas en SVG propio
Lector	`html5-qrcode`	Cámara del navegador para QR y barras
Publicación	Cloudflare Tunnel + `qrcode`	URL HTTPS temporal y QR de acceso

2. Arquitectura

El navegador pide la página y llama a la API con `fetch`. El servidor valida la sesión, ejecuta consultas SQL preparadas y responde JSON. La interfaz vuelve a pedir `/api/data` después de cada cambio y se redibuja.



Arquitectura general de Finax

3. Estructura del repositorio

```
finax/
├─ server.js            API REST (Express)
├─ package.json         dependencias y comandos
├─ .env.example         plantilla de variables (copiar a .env)
├─ .gitignore           excluye .env, node_modules, logs
├─ publicar.sh          túnel HTTPS + QR de acceso
├─ qr.js                genera el QR de la URL pública
├─ codigos.js           genera códigos de prueba para el lector
├─ public/              interfaz (index.html, logos, favicon)
├─ database/
│   ├─ schema.sql       esquema completo + datos iniciales
│   └─ migraciones/     para bases creadas con versiones anteriores
└─ docs/                manuales (PDF) y capturas
```

4. Base de datos

Ocho tablas relacionadas. `usuarios` y `categorias` son el centro: todo lo demás se enlaza con ellas. Al borrar un usuario se borran en cascada sus datos; al borrar una meta de ahorro se borran sus abonos.

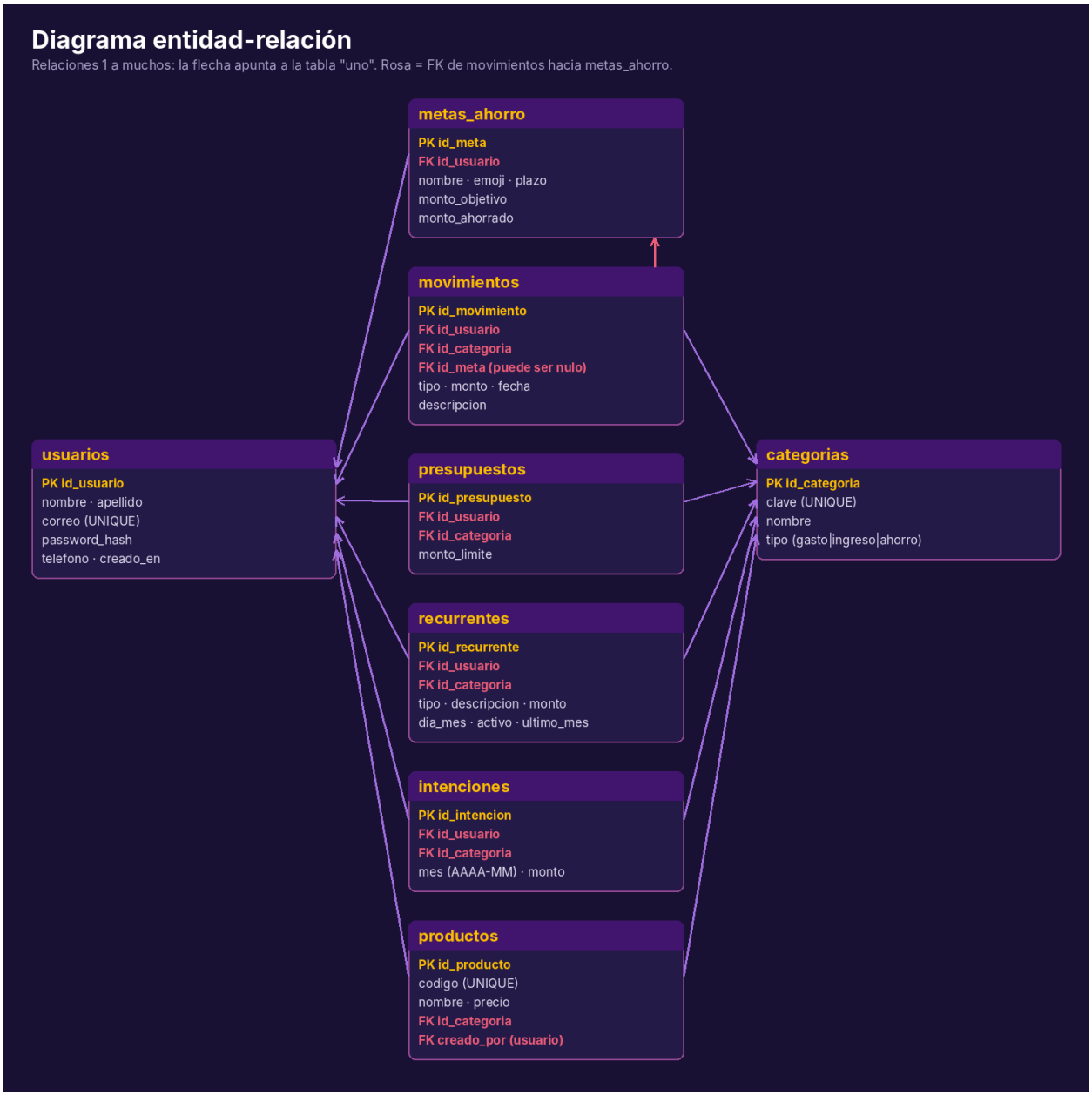


Diagrama entidad-relación

Diccionario de datos

usuarios

Personas registradas. La contraseña se guarda como hash bcrypt.

Columna	Tipo	Restricciones / uso
id_usuario	INT	PK, autoincremental
nombre, apellido	VARCHAR(50)	Obligatorios
correo	VARCHAR(100)	UNIQUE; identifica al usuario en el login
password_hash	VARCHAR(255)	Nunca texto plano
telefono	VARCHAR(20)	Opcional
creado_en	TIMESTAMP	Por defecto la fecha actual

categorias

Catálogo fijo con datos iniciales.

Columna	Tipo	Restricciones / uso
id_categoria	INT	PK
clave	VARCHAR(30)	UNIQUE (comida, transporte, ...)
nombre	VARCHAR(40)	Texto visible
tipo	ENUM	gasto, ingreso o ahorro

movimientos

Todo el dinero que entra y sale.

Columna	Tipo	Restricciones / uso
id_movimiento	INT	PK
id_usuario	INT	FK → usuarios (CASCADE)
id_categoria	INT	FK → categorias
id_meta	INT	FK → metas_ahorro (CASCADE), nulo salvo en abonos
tipo	ENUM	gasto, ingreso o ahorro
monto	DECIMAL(10,2)	CHECK > 0
fecha	DATE	Índice (id_usuario, fecha)
descripcion	VARCHAR(50)	Obligatoria

metas_ahorro

Metas con su avance.

Columna	Tipo	Restricciones / uso
id_meta	INT	PK
id_usuario	INT	FK → usuarios (CASCADE)
nombre, emoji	VARCHAR	Texto de la meta
plazo	ENUM	corto o largo
monto_objetivo	DECIMAL(10,2)	CHECK > 0
monto_ahorrado	DECIMAL(10,2)	Por defecto 0

presupuestos

Límite mensual por categoría.

Columna	Tipo	Restricciones / uso
id_presupuesto	INT	PK
id_usuario, id_categoria	INT	FK; UNIQUE juntos
monto_limite	DECIMAL(10,2)	CHECK > 0

recurrentes

Pagos o ingresos que se repiten cada mes.

Columna	Tipo	Restricciones / uso
id_recurrente	INT	PK
id_usuario, id_categoria	INT	FK
tipo	ENUM	gasto o ingreso
descripcion, monto	VARCHAR, DECIMAL	Monto > 0
dia_mes	TINYINT	Entre 1 y 31
activo	TINYINT(1)	1 = activo
ultimo_mes	CHAR(7)	Último mes atendido (AAAA-MM)

intenciones

Lo que la persona se propone cada mes (Mi mes en espejo).

Columna	Tipo	Restricciones / uso
id_intencion	INT	PK
id_usuario, id_categoria	INT	FK; UNIQUE con mes

Columna	Tipo	Restricciones / uso
mes	CHAR(7)	AAAA-MM
monto	DECIMAL(10,2)	En ahorro: mínimo a ahorrar. En gastos: máximo (0 = nada)

productos

Catálogo que usa el lector de códigos.

Columna	Tipo	Restricciones / uso
id_producto	INT	PK
codigo	VARCHAR(64)	UNIQUE; lo que lee la cámara
nombre, precio	VARCHAR, DECIMAL	Precio > 0
id_categoria	INT	FK → categorias
creado_por	INT	FK → usuarios (SET NULL)

5. Guía de clonación y despliegue desde GitHub

Requisitos

- Git, **Node.js 20 o superior** y **MySQL 8** instalados.
- Para la exposición: `cloudflared` (opcional, solo para la URL pública).

Pasos

1. **Clonar** el repositorio y entrar a la carpeta.
2. **Instalar** las dependencias.
3. **Crear la base de datos** con el script SQL.
4. **Configurar** el archivo `.env`.
5. **Iniciar** el servidor y abrir el navegador.

```
git clone https://github.com/TU_USUARIO/finax.git
cd finax
npm install
```

```
# crea la base "finax", las tablas, las categorías y los productos de ejemplo
sudo mysql < database/schema.sql
```

```
cp .env.example .env
# edita .env y escribe tu contraseña de MySQL y un JWT_SECRET largo.
# Puedes generar uno con:
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

```
npm start
# abre http://localhost:3000
```

Variable	Ejemplo	Significado
DB_HOST	localhost	Servidor MySQL
DB_USER	finax_app	Usuario de la base
DB_PASS	(tu clave)	Contraseña del usuario
DB_NAME	finax	Nombre de la base
JWT_SECRET	(64 caracteres)	Firma de sesiones; mínimo 20
PORT	3000	Puerto (opcional)

Usuario de MySQL

Se recomienda un usuario propio con permisos solo sobre la base `finax`:
`CREATE USER 'finax_app'@'localhost' IDENTIFIED BY 'tu_clave'; GRANT ALL ON finax.* TO 'finax_app'@'localhost'; FLUSH PRIVILEGES;`

Siempre activo (Linux, systemd)

```
[Unit]
Description=Finax
After=network.target mysql.service
[Service]
WorkingDirectory=/ruta/a/finax
ExecStart=/usr/bin/node server.js
Restart=always
User=TU_USUARIO
[Install]
WantedBy=multi-user.target
```

Guárdalo en `/etc/systemd/system/finax.service` y ejecuta `sudo systemctl enable --now finax`. Logs: `journalctl -u finax -n 25 --no-pager`.

Publicar en internet para la exposición

```
sudo systemctl stop finax     # si usas el servicio, libera el puerto 3000
bash publicar.sh              # abre el túnel HTTPS y muestra el QR
# Ctrl+C para cerrar
```

El script arranca Finax en modo público, abre un túnel de Cloudflare, obtiene una URL `https://....trycloudflare.com` y genera `qr-finax.png`. Quien escanee ese QR llega a la **pantalla de inicio de sesión** (con botón de demostración).

Bases creadas con versiones anteriores

Si ya tenías la base, ejecuta solo lo que falte de `database/migraciones/` (por ejemplo `sudo mysql finax < database/migraciones/migracion_productos.sql`). En una instalación nueva no hace falta: `schema.sql` ya lo incluye todo.

6. API REST

Todas las rutas (salvo registro, login y demo) exigen la cookie de sesión y devuelven JSON. Los errores devuelven `{ "error": "mensaje" }` con código 400, 401 o 500.

Método y ruta	Qué hace
POST /api/auth/register	Crea la cuenta (hash bcrypt) e inicia sesión
POST /api/auth/login	Valida correo y contraseña contra la base
POST /api/auth/demo	Crea una cuenta temporal con datos de ejemplo
POST /api/auth/logout · GET /api/auth/me	Cierra sesión · datos del usuario
PUT /api/profile · PUT /api/auth/password	Edita perfil · cambia contraseña
GET /api/data	Devuelve movimientos, metas, presupuestos, recurrentes e intenciones
POST · DELETE /api/tx	Crea / elimina un ingreso o gasto
POST · DELETE /api/goals · POST /api/goals/:id/contrib	Metas de ahorro y abonos
PUT · DELETE /api/budgets	Presupuestos por categoría
/api/recurring (POST, PUT, DELETE, toggle, skip, confirm)	Pagos recurrentes
PUT /api/intentions	Guarda las intenciones del mes actual
GET /api/products/:codigo	Busca un producto por el código leído
POST /api/products	Registra un producto nuevo

7. Lector de QR / código de barras

El lector está integrado a una función real: **registrar un gasto**. Desde Inicio o Gastos, el botón 📷 **Escanear** abre la cámara.

1. El navegador lee el código con `html5-qrcode` (también acepta escribirlo o un lector USB, que actúa como teclado).
2. La app llama a `GET /api/products/:codigo`.
3. **Si existe:** muestra nombre, categoría y precio; la persona elige cantidad y fecha y se crea el gasto en `movimientos`.
4. **Si no existe:** mensaje “Código no encontrado” y formulario para registrar el producto (`POST /api/products`); luego se guarda el gasto.
5. La alerta de presupuesto y Mi mes en espejo se actualizan solos porque usan los mismos movimientos.

Códigos de prueba

`node codigos.js` genera `codigos-prueba.html` con un QR y un código de barras por producto de ejemplo (FINAX-001 a FINAX-006), listos para imprimir o mostrar en otra pantalla.

Cámara y HTTPS

Los navegadores solo permiten la cámara en **https** o en `localhost`. Con el túnel de `publicar.sh` funciona en el celular; por la IP del wifi (http) no.

8. Seguridad

- **Contraseñas** con bcrypt (coste 12); nunca en texto plano ni en el código.
- **Sesión** con JWT de 2 horas en cookie `httpOnly` y `sameSite=strict` (`secure` en modo público).
- **Consultas preparadas** en todo el servidor: evita inyección SQL.
- **Validación** de cada dato en el servidor (tipos, rangos, longitudes y formatos).
- **Aislamiento:** cada consulta filtra por el `id_usuario` de la sesión.
- **Límite de intentos** en login, registro y demo (`express-rate-limit`).
- **Secretos** solo en `.env`, excluido de Git; se sube `.env.example` sin valores reales.
- Las cuentas de demostración se borran solas a las 24 horas.

9. Manejo de errores

Situación	Qué ocurre
Correo o contraseña incorrectos	Mensaje específico en el login; el servidor responde 400
Sesión vencida	401 y la app vuelve al inicio de sesión
Código no encontrado	Aviso y formulario para registrarlo
Cámara bloqueada	Aviso y campo para escribir el código
Dato inválido	400 con el motivo (monto, fecha, categoría…)
Fallo inesperado	500 “Error del servidor” y detalle en el log

10. Mi mes en espejo (lógica)

- Solo se puede declarar el **mes actual**; el servidor lo comprueba con la fecha de MySQL.
- En ahorro, el monto es un **mínimo**; en las categorías de gasto, un **máximo** (0 = nada).
- El resumen se calcula en el navegador con una función pura `mirror(mes, movimientos, intenciones, metas, hoy)`: cuenta cuántas intenciones se cumplieron (“Cumpliste X de Y”).

- Notas: cuánto se pasó, porcentaje de fin de semana (≥75 % “casi todo”, ≥55 % “la mayor parte”) y mayor gasto.
- Retraso de la meta: `round((S − A) / S × 30)` días, con S = ahorro propuesto y A = ahorro real.

11. Limitaciones y mejoras

- Es una app de **gestión personal**: no mueve dinero real ni se conecta a bancos.
- No hay recuperación de contraseña por correo.
- El catálogo de productos es compartido entre usuarios.
- La política de seguridad de contenido (CSP) está desactivada porque el JavaScript va dentro de `index.html`.
- Mejora futura: separar el JavaScript en archivos, activar CSP y añadir pruebas automáticas.